SECURITY & THREAT MODEL SPECIFICATION

Touch Dashboard Security Architecture

Authentication, Pairing Protocols, Network Isolation & Defense-in-Depth

Platform: Linux / Windows
Server: FastAPI / Uvicorn (Port 8888)
Transport: HTTP/1.1 & WebSockets
Verification: 32 Hermetic Security Tests

01 Executive Summary & LAN Threat Model

Touch Dashboard operates as a dual-surface system controller: a high-privilege Python backend running on the host workstation paired with touchscreens (Android tablets and browsers). The server binds to `0.0.0.0:8888` so handheld devices on the local network can connect seamlessly. Because local area networks cannot be treated as secure enclaves, Touch Dashboard operates under a **zero-trust LAN assumption**, actively mitigating four key threat vectors:

THREAT DNS Rebinding & CSRF
Hostile websites browsed on client machines rebind public domain names to `127.0.0.1` or host LAN IPs to bypass CORS boundaries and silently issue commands to local servers.

THREAT LAN Snooping & Impersonation
Unauthorized guest devices on shared Wi-Fi scan port 8888 to inject shell macros, synthesize virtual keystrokes, extract private API secrets, or snoop on desktop audio streams.

THREAT Side-Channel Timing Exploits
Variable-time cryptographic comparisons in token or pairing verification reveal valid token prefixes or database indices through statistical response latency analysis.

THREAT Local Multi-User Interference
On shared Unix/Windows systems, other unprivileged user accounts or local processes target loopback ports to trigger unauthorized shutdowns or inject IPC messages.

02 Network Boundary Defense & Connection Filtering

2.1 Host Header Guard (`_HostGuard` ASGI Middleware)

To defeat DNS rebinding before requests penetrate the application layer, the custom ASGI middleware `_HostGuard` intercepts every HTTP request and WebSocket handshake:

- **Strict Host Whitelist:** The `Host` header must match `127.0.0.1:<port>`, `localhost:<port>`, or one of the host PC's verified active LAN IPv4 addresses on the running port. Requests carrying foreign domains (e.g. `attacker.com:8888`) are immediately rejected with HTTP `403 Forbidden`.
- **Handshake Rejection:** WebSocket connections with invalid hosts receive close code `1008 (Policy Violation)` before the connection handshake completes.
- **Rate-Limited Dynamic Re-Scan:** When a request presents an unknown numeric IP literal on the dashboard's port (such as following a router DHCP renewal), the server refreshes local interface bindings at most once every 30 seconds (`LAN_REFRESH_S`), preventing lockout without introducing DoS risks.

2.2 WebSocket Origin Policy & Android System WebView Quarantine

The server validates incoming `Origin` headers via `OriginPolicy.origin_allowed()` :

ORIGIN VALUE	CLIENT CONTEXT	ASSIGNED ROLE	SECURITY ENFORCEMENT
`http://127.0.0.1:<port>` `http://localhost:<port>`	Loopback client (PC desktop window or browser)	ADMIN	Proven host peer; receives full configuration authority and direct control access.
`http://<lan-ip>:<port>`	LAN tablet or mobile browser	DEVICE	Requires pre-approved pairing token; restricted strictly to display and trigger actions.
`file://` `https://appassets.androidplatform.net`	Bundled Android APK local asset container	DEVICE	Bundled local assets inside Android WebView shell; standard device sandbox.
`Origin: null`	Android WebView from non-loopback LAN IP	DEVICE *(Never Admin)*	Accepted **only** if peer is non-loopback AND User-Agent matches `(Linux; Android ...; wv)` . Never elevated to admin.
Any unlisted origin	Foreign websites / malicious origins	CLOSED 1008	Rejected during WebSocket handshake; prevents cross-site hijacking.

2.3 Content Security Policy (CSP) & Response Hardening

All HTTP responses pass through `_SecurityHeaders` , injecting strict defense headers:

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' [unsafe-eval: loopback only];
                         style-src 'self' 'unsafe-inline'; img-src 'self' data: https:;
                         connect-src 'self' ws://<host>; object-src 'none'; base-uri 'none';
                         form-action 'none'; frame-ancestors 'none'
X-Frame-Options: DENY  |  X-Content-Type-Options: nosniff  |  Cache-Control: no-cache, must-revalidate
```

Conditional Evaluation Scoping: `pywebview` relies on `new Function` for its internal JS bridge on the desktop window. Touch Dashboard dynamically injects `'unsafe-eval'` *only* into responses served to `127.0.0.1` loopback. Pages served across the LAN never receive `'unsafe-eval'` , reducing attack surface on mobile clients.

03 Authentication & Cryptographic Pairing Protocol

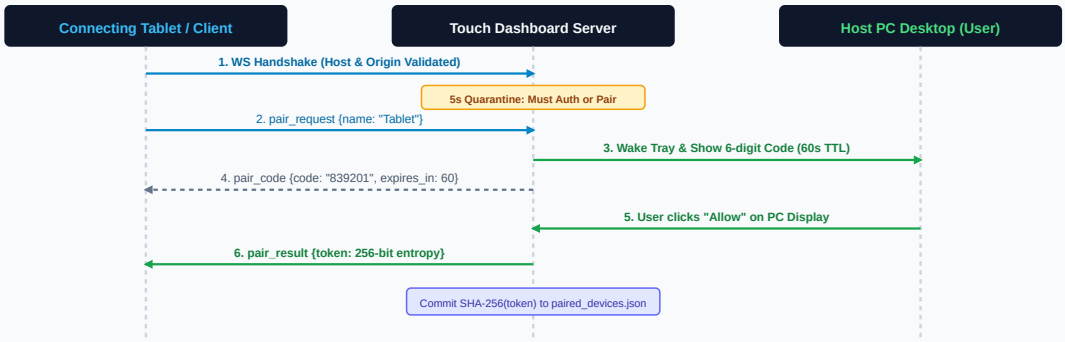


Figure 1: Two-Factor Physical Presence Verification & Zero-Knowledge Token Issuance Pipeline

3.1 Connection Quarantine Window (`AUTH_TIMEOUT_S = 5.0s`)

When a WebSocket connection opens, it is assigned to an unauthenticated socket pool. The client must transmit an `auth` or `pair_request` frame within 5.0 seconds. If this deadline lapses, or if unparsable JSON or unexpected message types are transmitted, the server delivers an update notice and severs the link with policy code `1008`.

3.2 Two-Factor Physical Presence Pairing Protocol

1. **Request Initiation:** Unpaired clients send `{"type": "pair_request", "device_name": "..."}`. The name is stripped of control characters and capped at 40 chars.
2. **Out-of-Band Physical Challenge:** The server generates a cryptographically random 6-digit numeric code (`secrets.randbelow(1_000_000)`) with a 60-second expiration. It wakes the desktop window from the system tray (`_desktop_module.wakeup()`) and displays the pending request.
3. **User Decision & Race Handling:** The PC owner verifies that the 6-digit code on the desktop matches the tablet. The server awaits an `asyncio.Future`. If the client disconnects or cancels, the pairing operation aborts cleanly.
4. **Token Issuance:** Upon user approval, the server generates a 256-bit URL-safe token (`secrets.token_urlsafe(32)`). The token is transmitted once over the authenticated WebSocket, after which the client caches it in persistent storage.

3.3 Zero-Knowledge Token Storage & Timing Attack Immunity

The host PC **never stores plaintext bearer tokens**. Instead, only the SHA-256 digest is written to `~/.config/touch-dashboard/paired_devices.json`.

> **CRYPTOGRAPHIC GUARD** **Constant-Time Verification Across Entire Database**
>
> When a paired device connects presenting a token, the server computes its SHA-256 hash and compares it against all registered devices using `hmac.compare_digest()`. Crucially, **the lookup loop never terminates early upon finding a match**; it scans every single record in the database. This guarantees that execution duration is mathematically independent of token validity or database position, neutralizing remote timing attacks.

04 Authorization, Access Control & Data Redaction

4.1 Centralized Role Dispatcher (`_dispatch()` & `_refusal()`)

Access control is enforced at a single gateway prior to executing any message handler in `WS_HANDLERS`:

PRIVILEGE TIER	GUARDED ACTIONS / COMMANDS	SECURITY POLICY
Administrative Operations	`save_config`, `save_macros`, `set_theme`, `set_play_key`, `check_update`, `run_update`, `pair_decision`, `rename_device`, `revoke_device`	ADMIN ONLY Paired devices receive `{"type": "refused", "data": {"reason": "admin_only"}}`.
Credential Management	`action:spot_clear_auth`, `action:disc_auth`, `action:disc_clear_auth`	ADMIN ONLY Prevents remote revocation or token interception.
AV Controls	`action:disc_cam`, `action:disc_screen` (Discord camera & screen share toggles)	CONFIGURABLE Requires `device_av_controls: true` on the host workstation.
Execution by ID	`run_macro {id}`, `play_clip {id}`	PAIRED DEVICES Invokes server-defined IDs; clients cannot submit shell strings.

4.2 Configuration Allowlisting & Secret Isolation

Paired devices never receive the workstation's raw `config.json` file. All state broadcast to devices is sanitized via `_device_config_sync()` using explicit field allowlists (`DEVICE_CFG_KEYS` and `DEVICE_SYNC_KEYS`):

- **Excised Secrets:** OpenWeatherMap API keys, Spotify client IDs & secrets, Discord OAuth tokens & client secrets, host soundboard directory paths (`sounds_path`), and raw shell execution strings are omitted entirely.
- **Opaque Macro Identifiers:** Macro definitions delivered to devices contain only display metadata: `{id, display_name, icon_url}`. To trigger an action, the device sends `run_macro {id: "..."}`. The server validates the ID against `ITEM_ID_RE = re.compile(r"[A-Za-z0-9_-]{1,64}")` and resolves the corresponding executable on the host machine.

4.3 Filesystem Path Traversal & Command Injection Defenses

DEFENSE **Realpath Sound Traversal Check**

`resolve_sound_path()` normalizes paths with `os.path.realpath` and verifies that `candidate.startswith(base + os.sep)`. Directory traversal sequences (`../`) cannot escape the configured sounds folder.

DEFENSE **Soundpad Parameter Sanitization**

Windows Soundpad executes pad actions via `-rc` command line switches. Pad indexes are strictly constrained to numeric digits via `re.compile(r"[0-9]{1,6}")`, preventing CLI command injection.

05 Local IPC, Media Signatures & OAuth Security

5.1 Ephemeral Local Control Token (`LocalControlToken`)

The host endpoints `/api/wakeup` (bringing GUI to front) and `/api/exit` (installer shutdown) are reachable from loopback. Because multiple local users or web browsers share loopback:

- At startup, the server generates a 256-bit random secret stored in `local_control.token` (POSIX mode `0600`).
- Requests require three conditions: (1) Loopback peer address, (2) `Content-Type: application/json` (which browsers cannot send cross-origin without preflight), and (3) matching `Authorization: Bearer <token>` checked via `hmac.compare_digest`.

5.2 Ephemeral HMAC-SHA256 Media Art Signing

Standard HTML `<img>` tags cannot transmit custom authorization headers. To prevent unauthenticated network observers from harvesting cover art:

```
# Server generates random 256-bit key at startup
_ART_KEY = secrets.token_bytes(32)

def _sign_art(h: str) -> str:
    return hmac.new(_ART_KEY, h.encode(), "sha256").hexdigest()[:32]
```

Cover URLs sent to clients are formatted as `/api/local_art?h=<hash>&k=<signature>`. The endpoint verifies both that the signature matches and that h matches the **currently active track**. Once a song ends or playback changes, older image URLs are immediately invalidated.

5.3 CSRF-Resistant OAuth State Tokens

Spotify and Discord OAuth login flows require authentication on the host PC. Authorization URLs embed cryptographically secure single-use `state` tokens with a 10-minute TTL. The redirect endpoints (`/callback` and `/disc_callback`) are loopback-only and atomically consume the state token via `_spend_oauth_state()`.

06 Denial-of-Service & Storage Integrity

6.1 Sockets & Rate Limits

GUARD LAYER	THRESHOLD	MITIGATED RISK
`PendingSockets`	32 total / 4 per IP	Exhaustion of connection threads & tasks prior to auth.
`PairingRequests`	5 per min / 1 per IP	Prompt flooding and UI denial-of-service on desktop.
`WS_MAX_MESSAGE_BYTES`	1 MiB frame limit	Memory blowup (lowered from Uvicorn's 16 MiB default).
Token Input Bounds	Max 256 characters	CPU exhaustion via oversized hashing inputs.

6.2 Atomic File I/O & Permissions

Atomic Write-Then-Swap Pattern

Critical files (`paired_devices.json` , `config.json` , `spotify_token.json` , `local_control.token`) are written to temporary staging files (`<file>.tmp`) using POSIX file descriptor mode `0600` (read/write by owner only). They are committed using atomic `os.replace()` to ensure zero partial-write corruption and prevent multi-user eavesdropping.

07 Mobile App & Update Integrity

- **Cloud Backup Shield:** The Android manifest explicitly declares `android:allowBackup="false"`. Paired device bearer tokens stored in the WebView's local storage never leave the tablet in automated Google cloud backups.
- **Verified Update Pipeline:** Desktop installers and the Android self-updater (`AppUpdater.java`) fetch binaries exclusively from GitHub releases over HTTPS. The payload is checked against the release's `SHA256SUMS` manifest via `hmac.compare_digest` prior to execution.
- **Shell-Free Execution:** Desktop installers execute via `asyncio.create_subprocess_exec` (direct binary execution without shell interpolation). Android updates are delivered through an unexported `UpdateFileProvider` granting scoped read permissions solely to the system `PackageInstaller` .

08 Security Verification Matrix

Touch Dashboard maintains a comprehensive automated security test suite in `tests/test_security.py`. All 32 security tests execute hermetically on every commit and release build:

TEST CASE IDENTIFICATION	TARGET VULNERABILITY / THREAT	MECHANISM VERIFIED	SUITE STATUS
`test_bad_host_gets_403`	DNS rebinding across arbitrary domain names	`_HostGuard` ASGI filter	PASS Verified
`test_foreign_websocket_origin_is_closed`	Cross-site WebSocket hijacking from third-party sites	`OriginPolicy` check	PASS Verified
`test_tablet_null_origin_accepted_only_lan`	Malicious iframe impersonating Android WebView `null`	User-Agent & IP checks	PASS Verified
`test_no_auth_within_five_seconds_closes`	Connection slot hoarding and unauthenticated task leak	`AUTH_TIMEOUT_S` drop	PASS Verified
`test_device_token_cannot_run_admin_actions`	Privilege escalation from paired tablet to admin actions	`_refusal()` gatekeeper	PASS Verified
`test_config_sent_to_device_has_no_secrets`	Extraction of Spotify/Discord/Weather secrets by tablet	`_device_config_sync()`	PASS Verified
`test_run_macro_runs_by_id_rejects_commands`	Arbitrary shell command or script injection over network	`ITEM_ID_RE` & ID lookup	PASS Verified
`test_local_control_routes_need_token`	Local privilege abuse of `/api/exit` or `/api/wakeup`	Bearer + 0600 token	PASS Verified
`test_local_art_needs_signed_url_current_cover`	Cover art scraping and historical URL enumeration	HMAC-SHA256 signature	PASS Verified
`test_update_installer_must_match_checksum`	Man-in-the-middle tampering of auto-update scripts	`SHA256SUMS` constant-time	PASS Verified

Defense Summary: Zero-Trust LAN Assurance

By enforcing strict origin filtering, zero-knowledge token hashing, constant-time database matching, full configuration redaction, and isolated local control tokens, Touch Dashboard provides a hardened, commercial-grade security model suitable for exposed multi-device home and office networks.